TransferLab

End-to-end .NET, Azure, Kubernetes, migration, testing, and portfolio lab

Purpose	Build a working transaction-processing prototype locally, migrate it to Azure, deploy it to Kubernetes, and docu
Primary stack	C# / .NET 8, ASP.NET Core, EF Core, SQL Server, React + TypeScript, Docker, Azure SQL, AKS, GitHub Acti
Portfolio angle	A documented migration case study rather than a generic CRUD demo.
Interview angle	Hands-on practice for C#/.NET, cloud migration, Kubernetes, testing, code review, debugging, CI/CD, and prod

Guiding principle: Separate infrastructure migration risk from application modernization risk wherever practical.

Prepared as a living project document. Update it as implementation decisions change.

Contents

1. Project purpose and success criteria
2. Functional prototype scope
3. Local / on-premises-style architecture
4. Azure target architecture
5. Repository and documentation strategy
6. Implementation roadmap
7. Migration plan
8. Testing and validation strategy
9. Kubernetes and AKS practice plan
10. CI/CD and observability
11. Failure drills and rollback
12. Portfolio / interview narrative
13. Stretch goals and technical glossary

What makes this project useful

TransferLab is intentionally small in business scope and broad in engineering scope. The business rules are simple enough to understand completely, which leaves room to practice the infrastructure and migration concerns that matter in a senior .NET / Azure interview: dependency discovery, containerization, cloud database migration, Kubernetes deployment, health checks, test automation, observability, cutover criteria, failure recovery, and rollback.

- **Not a banking product:** all accounts and transactions are synthetic.
- **Not a pure cloud-greenfield app:** the first version intentionally behaves like a locally hosted service.
- **Not a one-shot deployment:** the project preserves a before/after migration story and documents why each change is made.
- **Not merely a portfolio UI:** the strongest artifact is the architecture, migration plan, testing evidence, and operational runbook.

1. Project Purpose and Success Criteria

TransferLab simulates a simple account-to-account funds transfer platform. A user can create synthetic accounts, submit transfers, inspect status, and view transaction history. The system will be implemented locally first, then migrated incrementally into Microsoft Azure.

Core learning objectives

- Refresh hands-on C# and modern .NET skills without relying on AI-assisted coding during critical implementation blocks.
- Build ASP.NET Core APIs with dependency injection, asynchronous code, EF Core, validation, and transactional data handling.
- Use SQL Server locally and Azure SQL in the cloud.
- Containerize the API and background worker using Docker.
- Deploy the containers to Azure Kubernetes Service and troubleshoot pods, services, probes, configuration, and rollouts.
- Create automated unit, integration, smoke, and migration-validation tests.
- Use GitHub Actions for repeatable build, test, image, and deployment workflows.
- Instrument the system for logs, health, latency, dependency failures, and post-deployment validation.
- Practice a migration sequence with explicit acceptance criteria and rollback capability.

Definition of done

Area	Evidence
Application	Working UI, API, worker, and persisted synthetic data
Local deployment	Docker Compose launches the full local stack
Cloud data	Application successfully uses Azure SQL
Kubernetes	API and worker run in AKS with health probes and repeatable manifests
Testing	Automated unit, integration, API, smoke, and migration checks
CI/CD	Push triggers build/test; deployment workflow can publish and roll out images
Operations	Logs and telemetry support troubleshooting; documented failure drills completed
Documentation	README, architecture docs, ADRs, migration plan, testing strategy, runbook, diagrams

2. Functional Prototype Scope

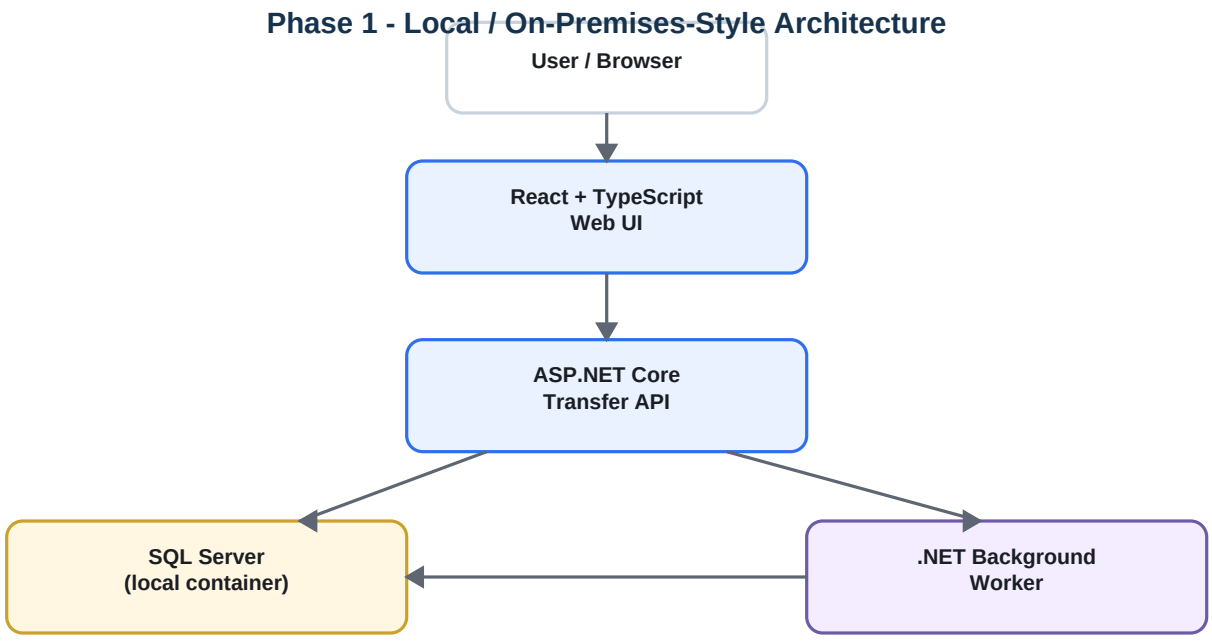
The prototype intentionally uses a small domain so that infrastructure work remains visible.

Capability	Proposed endpoint / behavior
Create account	POST /api/accounts
Account lookup	GET /api/accounts/{id}
Submit transfer	POST /api/transfers
Transfer status	GET /api/transfers/{id}
Transaction history	GET /api/accounts/{id}/transactions
Liveness	GET /health/live
Readiness	GET /health/ready

Key business rules

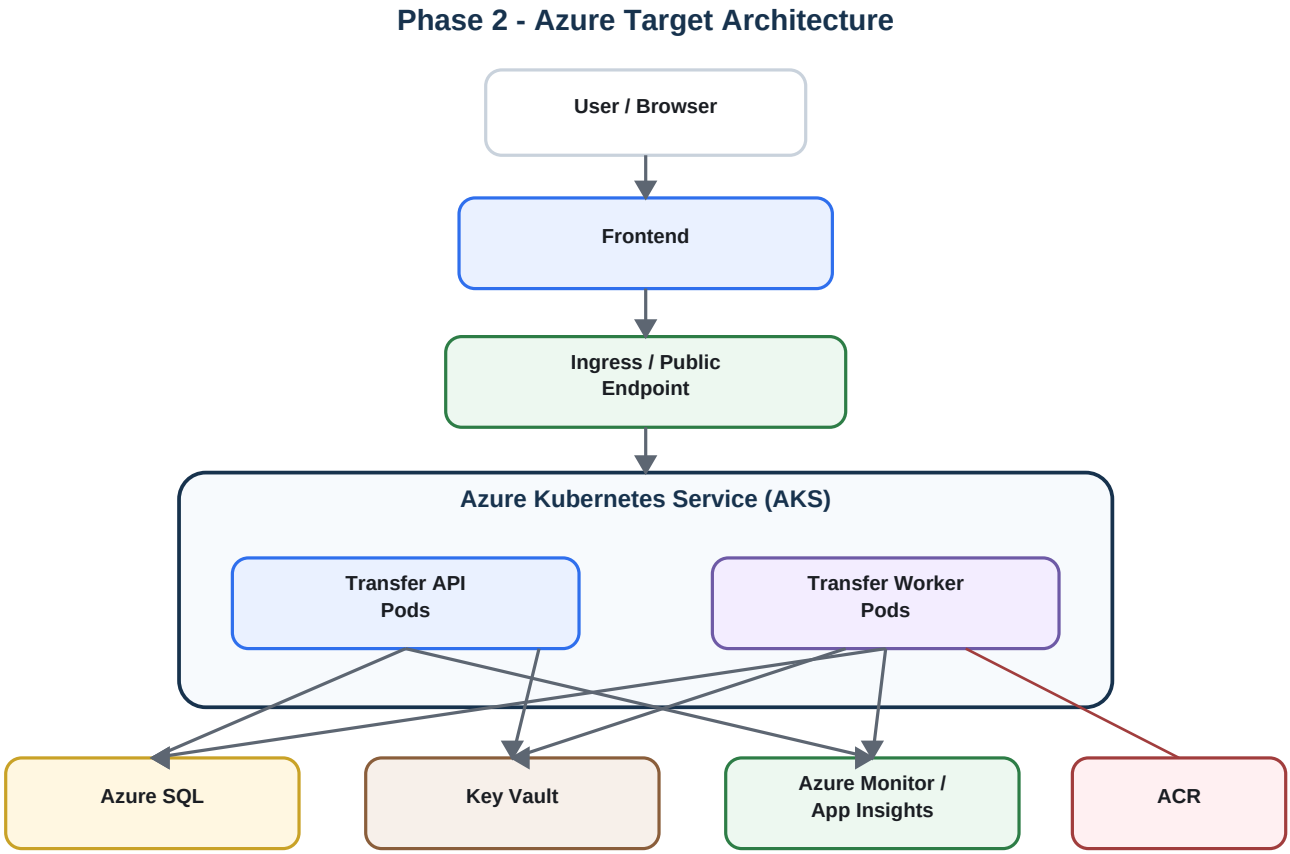
- A transfer must have a positive amount.
- Source and destination accounts must exist and must differ.
- A source account cannot transfer more than its available balance.
- Debit and credit must be committed atomically.
- Transfer creation must support an **Idempotency-Key** so retries cannot duplicate a financial operation.
- A background worker processes pending transfers and records Completed or Failed status.

3. Local / On-Premises-Style Architecture



The local version is the behavioral baseline. It should be working, tested, and tagged before cloud migration begins. The goal is to know what correct behavior looks like before infrastructure changes introduce additional failure modes.

4. Azure Target Architecture



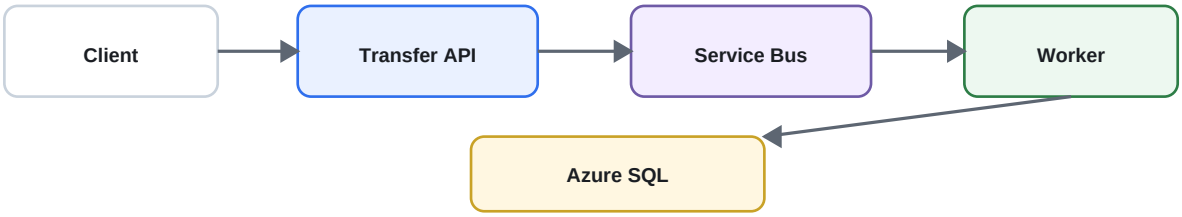
The target state moves persistence to Azure SQL, application runtime to AKS, container storage to Azure Container Registry, secrets to Key Vault, and telemetry to Azure Monitor / Application Insights. Managed Identity should be preferred over embedded credentials where practical.

Service mapping

Local / traditional concern	Azure target
IIS / local service runtime	AKS (or App Service / Container Apps for comparison)
SQL Server	Azure SQL Database; evaluate Managed Instance if compatibility requires it
Local configuration / secrets	Environment configuration + Key Vault
File-based or shared storage	Azure Blob Storage / Azure Files if introduced
Background work	Worker deployment; later Service Bus for decoupling
Container images	Azure Container Registry
Monitoring	Azure Monitor / Application Insights; Datadog optional
Authentication to Azure services	Managed Identity where supported

Future asynchronous messaging

Future Async Messaging Architecture



Service Bus is deliberately a later phase. First establish a correct synchronous/worker flow. Then introduce messaging to explore retries, duplicate delivery, idempotent consumers, eventual consistency, dead-letter queues, and transient failure handling.

5. Repository and Documentation Strategy

Documentation is part of the product. The repository should make sense to a reviewer before they run the code.

```
transferlab/
■■■ src/
■   ■■■ TransferLab.Api/
■   ■■■ TransferLab.Worker/
■   ■■■ transferlab-web/
■■■ tests/
■   ■■■ TransferLab.UnitTests/
■   ■■■ TransferLab.IntegrationTests/
■■■ docker/
■■■ k8s/
■■■ docs/
■   ■■■ architecture.md
■   ■■■ migration-plan.md
■   ■■■ testing-strategy.md
■   ■■■ runbook.md
■   ■■■ adr/
■■■ .github/workflows/
■■■ README.md
```

Architecture Decision Records (ADRs)

Use ADRs for decisions where a future reviewer could reasonably ask, "Why did you do it this way?" Each ADR should contain Context, Decision, Alternatives, Rationale, and Consequences.

- 0001 - application architecture and project boundaries
- 0002 - SQL Server / Azure SQL persistence strategy
- 0003 - Docker containerization strategy
- 0004 - AKS deployment decision
- 0005 - idempotency implementation
- 0006 - asynchronous messaging decision, if Service Bus is added

Documentation rule

> **Update the documentation when architecture changes. Do not let diagrams describe an architecture the repository no longer implements.**

6. Implementation Roadmap

Day / Phase	Primary outcome	Hands-on focus
1 - Domain + API	Solution skeleton, entities, DbContext, initial endpoints	C# records/classes, DI, async, EF Core, API design
2 - Transfer engine	Transactional transfer logic, idempotency, unit/integration tests	Transactions, concurrency, validation, error handling
3 - UI + local stack	React UI, Dockerfiles, Docker Compose, end-to-end local flow	React/TS workflow, containers, configuration, networking
4 - Azure data	Azure account/resources, Azure SQL, connection config, migrations	Azure CLI, database migration, secrets, connectivity
5 - AKS	ACR, manifests, deployments, services, probes, hosted app	kubectl, pods, services, logs, health, rollout
6 - CI/CD + telemetry	GitHub Actions, automated validation, monitoring	Build/test/deploy pipeline, App Insights/Azure Monitor
7 - Failure drills + polish	Rollback, break/fix exercises, README and runbook refinement	Operational reasoning, interview rehearsal, portfolio polish

Implementation discipline

- Keep critical C# practice blocks AI-free so the project also refreshes interview-ready coding fluency.
- Commit at meaningful milestones with descriptive messages.
- Do not introduce cloud services before the local behavior is stable and testable.
- Prefer small changes that preserve rollback over broad rewrites.
- Capture screenshots, commands, and troubleshooting notes only when they teach something useful; avoid documentation noise.

7. Migration Plan

Migration Sequence



Phase	What to do	Artifacts / evidence
Discover	Inventory runtime, DB, config, secrets, networking, storage, auth, background jobs, and external dependencies.	Discovery notes
Baseline	Capture known-good API behavior, critical tests, representative data, latency, and failure scenarios.	Test results + baseline
Containerize	Build reproducible API/worker images; remove machine-specific assumptions.	Dockerfiles + Compose
Migrate data	Provision Azure SQL, apply schema, validate connectivity, compare data and queries.	Migration and reconciliation notes
Deploy	Push image to ACR, deploy manifests to AKS, configure secrets/config, and expose the app.	Config templates + deployment manifests
Validate	Run smoke/integration/contract tests; inspect errors, latency, health, and logs.	Acceptance checklist
Cut over	Direct traffic to target only after thresholds are satisfied.	Cutover checklist
Observe	Watch post-cutover telemetry and data correctness during a defined validation window.	Validation monitoring notes

Migration questions the project should answer

- What dependencies would prevent a direct move to a managed Azure service?
- What changes are infrastructure-only versus application modernization?
- How do we prove the new environment behaves like the old one?
- How do we keep secrets out of source code and images?
- How do we deploy a new version without immediately sending production traffic to an unhealthy pod?
- What are the rollback points for application, infrastructure, and data?

8. Testing and Validation Strategy

Testing is part of the migration, not a cleanup task after deployment. The project should establish evidence before migration, then reuse that evidence against the target environment.

Layer	Purpose	Examples
Unit	Validate isolated business rules	Amount validation, balance checks, status transitions, idempotency rules
Integration	Validate application + persistence behavior	DbContext, SQL transaction boundaries, dependency injection
API contract	Preserve externally visible behavior	Status codes, request/response shapes, error behavior
Database	Verify migrated persistence	Schema, constraints, row counts, indexes, reconciliation
Smoke	Verify critical target-system paths	Health, account lookup, transfer submission, transfer retrieval
Performance	Compare target to baseline	Latency, throughput, query duration, error rate
Migration acceptance	Decide whether cutover is safe	All critical checks + monitoring + rollback readiness

Recommended migration acceptance checklist

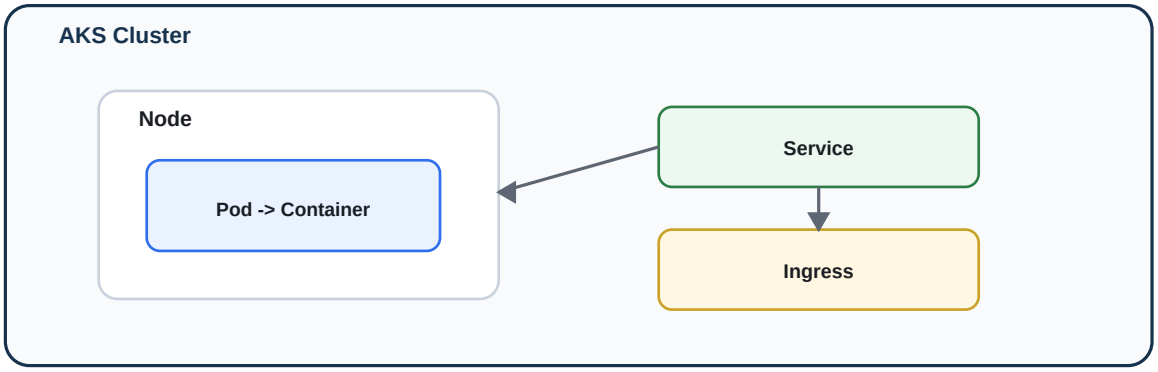
- Critical automated tests pass in the target environment.
- API contracts remain compatible.
- Representative data is reconciled correctly.
- Readiness and liveness probes remain stable.
- No unexpected spike in error rate or dependency failures.
- Latency is within a predefined acceptable threshold.
- Logs and telemetry are available before traffic is cut over.
- A tested rollback path still exists.

Idempotency test worth demonstrating

Send the same transfer request twice with the same Idempotency-Key. The second request must return the existing transfer result rather than executing another debit/credit. This creates a concrete discussion around retries, network ambiguity, duplicate delivery, and safe distributed operations.

9. Kubernetes and AKS Practice Plan

Kubernetes Concepts Used in the Lab



Deployments manage desired pod state; probes govern traffic and restarts.

Concepts to understand through use

- **Deployment:** desired state, replica count, rolling updates, rollback history.
- **Pod:** smallest deployable Kubernetes unit used here.
- **Service:** stable network identity for a changing set of pods.
- **Ingress:** inbound HTTP routing into the cluster.
- **ConfigMap:** non-sensitive runtime configuration.
- **Secret:** sensitive configuration; in Azure, prefer integration with Key Vault / Managed Identity.
- **Readiness probe:** determines whether a pod should receive traffic.
- **Liveness probe:** determines whether a stuck container should be restarted.
- **Resource requests/limits:** scheduler expectations and runtime boundaries.
- **Horizontal Pod Autoscaler:** optional stretch exercise for scaling behavior.

Commands to become comfortable with

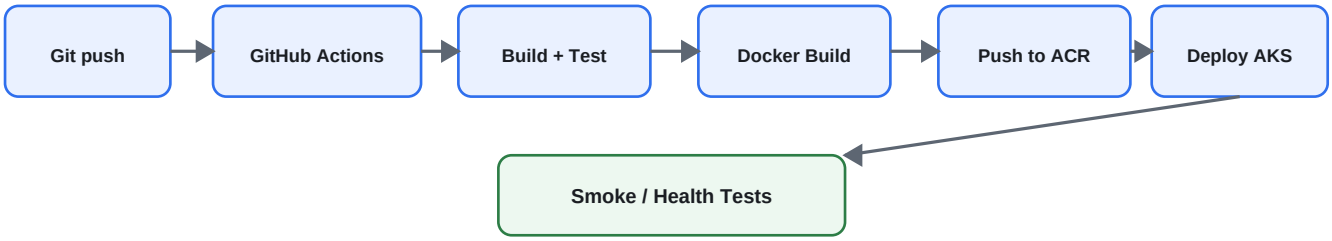
```
kubectl get pods
kubectl get deployments
kubectl get services
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl rollout status deployment/transfer-api
kubectl rollout undo deployment/transfer-api
kubectl scale deployment transfer-api --replicas=3
```

Troubleshooting sequence

Deployment status -> pod state -> events -> logs -> readiness/liveness -> service routing -> configuration/secrets -> database/dependency connectivity.

10. CI/CD and Observability

CI/CD Target Flow



Pipeline goals

- Build the solution on every pull request / main-branch change.
- Run unit and integration tests before packaging.
- Build versioned Docker images.
- Push deployable images to Azure Container Registry.
- Deploy manifests or update image tags in AKS.
- Wait for rollout status.
- Run smoke and health checks after deployment.
- Fail visibly when the rollout or smoke tests fail.

Observability signals

Signal	Why it matters during migration
Structured application logs	Correlate failures and compare behavior between environments
HTTP request rate / error rate	Detect cutover regressions quickly
Latency	Compare cloud target to local baseline
Dependency telemetry	Identify DB or downstream-service failures
Readiness / liveness	Separate app-process health from traffic-serving capability
Kubernetes events	Reveal scheduling, image, probe, configuration, and restart problems
Database metrics	Detect expensive queries, connection issues, or resource pressure

11. Failure Drills and Rollback

Intentionally breaking the system is one of the most valuable parts of the lab. The goal is to create operational muscle memory, not just a successful happy-path deployment.

Drill	Expected investigation / recovery
Bad database connection	Observe readiness failure, inspect pod logs/configuration, correct secret/config, verify recovery
Delete an API pod	Watch Deployment replace it; confirm Service continues routing to healthy replicas
Broken application image	Observe failed rollout, inspect events/logs, perform rollout undo
Failing readiness probe	Confirm pod stays alive but receives no traffic; fix dependency / probe condition
Failing liveness probe	Observe container restarts; distinguish restart loop from readiness failure
Database latency / failure simulation	Observe error/latency telemetry; confirm retry/error handling does not duplicate transfers
Duplicate transfer request	Verify idempotency prevents double processing
Bad schema / migration mismatch	Stop cutover; restore compatibility or roll back before accepting traffic

Rollback philosophy

> **Rollback is not merely a Kubernetes command. Treat application version, infrastructure state, configuration, and database state as separate rollback concerns. A migration should not be considered safe until the team knows what can be reversed and what cannot.**

12. Portfolio and Interview Narrative

The project should be discussed as a **self-directed migration lab**, not as prior enterprise migration ownership. The value is that the implementation creates fresh hands-on evidence and a concrete framework for discussing migration decisions.

Short portfolio description

> **TransferLab is a documented .NET cloud-migration lab that begins as a locally hosted ASP.NET Core and SQL Server transaction service, then moves persistence to Azure SQL and containerized services to Azure Kubernetes Service. The repository documents testing baselines, Kubernetes health probes, CI/CD, observability, idempotency, cutover criteria, failure drills, and rollback strategy.**

Interview-ready talking points

- I established known behavior before changing infrastructure.
- I separated migration concerns from modernization wherever possible.
- I treated testing and observability as migration prerequisites rather than post-deployment work.
- I implemented liveness and readiness separately so Kubernetes could distinguish process health from traffic readiness.
- I practiced rolling deployment and rollback rather than treating deployment as a one-way operation.
- I used idempotency to make retries safe across ambiguous network failures.
- I intentionally broke the environment and troubleshot from deployment -> pod -> events -> logs -> probes -> service routing -> dependencies.
- I documented architectural decisions and their consequences with ADRs.

What not to claim

Do not represent this lab as production banking software or as company-wide migration leadership. Present it as deliberate hands-on work that complements prior professional experience with .NET, Azure, AKS, Docker, production support, CI/CD, and observability.

13. Stretch Goals and Technical Glossary

Stretch goals - only after the core system works

- Azure Service Bus with retry and dead-letter handling.
- Outbox pattern to coordinate database state and message publication.
- Optimistic concurrency control for account balances.
- Horizontal Pod Autoscaler.
- Blue/green or canary-style deployment experiment.
- Azure API Management in front of the API.
- OpenTelemetry exporter and/or Datadog integration.
- Terraform or Bicep for infrastructure-as-code after the manual Azure CLI path is understood.
- Load testing with a small reproducible script and before/after comparison.
- Security review: least privilege, secret rotation, network boundaries, image scanning.

Glossary

Term	Working definition
Rehost	Move a workload with minimal application change, often to VM-like infrastructure.
Replatform	Change the hosting platform while keeping the core application largely intact.
Refactor / modernize	Change application architecture or code to take advantage of cloud-native patterns.
AKS	Azure Kubernetes Service - Microsoft-managed Kubernetes control plane with Azure-integrated compute/networking.
ACR	Azure Container Registry - private registry for container images.
Azure SQL	Managed SQL Server-compatible database service in Azure.
Managed Identity	Azure identity assigned to a resource so it can authenticate without embedded credentials.
Idempotency	Property that repeating the same operation does not create additional side effects.
Readiness	Whether an instance should receive traffic.
Liveness	Whether an instance should continue running or be restarted.
Smoke test	Small critical-path validation immediately after deployment.
Cutover	Transition of production traffic or responsibility from source environment to target.
Rollback	Return to a previously known-good application/infrastructure state after failed change.
ADR	Architecture Decision Record documenting context, decision, alternatives, rationale, and consequences.

Immediate next step

Finish repository initialization and documentation, create the first commit, create the GitHub repository over SSH, then begin Day 1 by generating the .NET solution and domain model from an empty directory.